

Video Script: Introduction to Seccomp

Module 3, Video 1 of 3

Estimated Duration: 12-14 minutes

Technical Setup: 1920x1080 @ 30fps, 150% zoom on all screens

Pre-Production Checklist

- [] PowerPoint slides (5 slides) ready
 - [] Terminal with large font (20-22 lines visible), dark theme
 - [] Docker running with test containers
 - [] Sample vulnerable container image built
 - [] Code editor (VS Code) with JSON syntax highlighting
 - [] Browser tabs: Docker docs, Linux syscalls reference
-

VIDEO SCRIPT

SEGMENT 1: Hook and Context (0:00 - 1:30)

[SCREEN: PPT Slide 1 - Title slide with dark background and code-themed graphics]

NARRATION:

“Welcome to Module 3: Seccomp Profiles in Practice. In the previous modules, we saw how containers can be exploited through kernel vulnerabilities, privileged access, and capability abuse. But what if we could prevent containers from even making dangerous system calls in the first place? That’s exactly what Seccomp does, and it’s one of the most powerful security mechanisms available for containers.

In this video, we’ll understand what Seccomp is, how it works at the kernel level, and analyze Docker’s default Seccomp profile. By the end, you’ll be able to view active profiles and test their restrictions hands-on.”

[TRANSITION: Fade to PPT Slide 2]

SEGMENT 2: What is Seccomp? (1:30 - 4:00)

[SCREEN: PPT Slide 2 - Diagram showing Linux kernel with syscalls filtering layer]

NARRATION:

“Seccomp stands for Secure Computing Mode. It’s a Linux kernel feature that allows you to filter system calls—the interface between user space applications and the kernel.

Think of it this way: every time a program wants to do something significant—open a file, create a network connection, change process permissions—it makes a system call to the kernel. The kernel has over 300 system calls available, but most applications only need a small subset.

[POINTER HIGHLIGHT: syscall layer in diagram]

Seccomp sits right at this boundary. When a process tries to make a syscall, Seccomp checks against an allowed list. If the syscall is permitted, it proceeds normally. If it's blocked, the call is rejected, and the process can be killed or simply get an error.

[TRANSITION: PPT Slide 3 - Three-column comparison]

This is critical for containers because if an attacker compromises your application, they're still constrained by the Seccomp profile. They can't make syscalls to mount filesystems, load kernel modules, or use advanced kernel features—even if they gain root inside the container."

[SCREEN: Terminal - maximize window]**NARRATION:**

"Let me show you this in action. I have a simple container here. Let's start it without any Seccomp restrictions."

[TYPE COMMAND - pause at each word for visibility]:

```
docker run --rm --security-opt seccomp=unconfined -it ubuntu:22.04 bash
```

[COMMAND EXPLANATION while typing:]

"The `--security-opt seccomp=unconfined` flag tells Docker to disable Seccomp entirely. This is dangerous in production, but useful for understanding what Seccomp prevents."

[PRESS ENTER, wait for container prompt]

SEGMENT 3: Understanding System Calls (4:00 - 6:30)**[SCREEN: Terminal - inside container]****NARRATION:**

"Now we're inside the container with no Seccomp restrictions. Let's see what syscalls we can make. First, let's check what happens when we try to create a new user namespace."

[TYPE COMMAND:]

```
unshare --user /bin/bash
```

[PRESS ENTER - command succeeds]**NARRATION:**

"That worked. User namespaces can be exploited for privilege escalation in certain scenarios. Now let's exit this container and start a new one WITH Docker's default Seccomp profile."

[TYPE:]

```
exit
```

[BACK TO HOST TERMINAL]

[TYPE COMMAND:]

```
docker run --rm -it ubuntu:22.04 bash
```

NARRATION:

“No special flags this time—Docker automatically applies its default Seccomp profile.”

[PRESS ENTER, wait for container prompt]

[TYPE COMMAND:]

```
unshare --user /bin/bash
```

[PRESS ENTER - command fails with error]

NARRATION:

“Notice the difference? We get ‘Operation not permitted’. The Seccomp profile blocked the `unshare` syscall. The application tried to make the call, but the kernel—through Seccomp—rejected it before it could execute.

This is the power of Seccomp: reducing the attack surface by blocking dangerous syscalls entirely.”

SEGMENT 4: Docker’s Default Seccomp Profile (6:30 - 9:30)

[SCREEN: Code Editor - VS Code with Docker default seccomp profile JSON open]

NARRATION:

“Let’s look at Docker’s default Seccomp profile to understand what it’s actually doing. This is a JSON file that defines which syscalls are allowed or blocked.”

[SCROLL to top of file slowly]

NARRATION:

“At the top, we see the profile structure. The `defaultAction` is set to `SCMP_ACT_ERRNO`. This means: by default, if a syscall is NOT in our allowed list, it returns an error.”

[HIGHLIGHT the line:]

```
"defaultAction": "SCMP_ACT_ERRNO",
```

NARRATION:

“Then we have an `architectures` array. Seccomp needs to know which CPU architectures this profile supports—x86_64, ARM, etc.”

[SCROLL DOWN to syscalls section]

NARRATION:

“Here’s the important part: the `syscalls` array. Each entry defines syscalls and their action.”

[HIGHLIGHT a syscall entry:]

```
{
  "names": [
    "accept",
    "accept4",
    "access",
    "adjtimex",
    "alarm"
  ],
  "action": "SCMP_ACT_ALLOW"
}
```

NARRATION (line by line):

- `names` : array of syscall names to apply this rule to
- `action` : what to do when these syscalls are invoked
- `SCMP_ACT_ALLOW` means: let these through

So these five syscalls—accept, accept4, access, adjtimex, and alarm—are permitted.”

[SCROLL DOWN to show blocked syscalls]

NARRATION:

“Now let’s find some interesting blocked syscalls. I’ll search for ‘mount’.”

[CTRL+F, search for “mount”]

NARRATION:

“You’ll notice `mount` is NOT in the allowed list, which means it will trigger the default action: `SCMP_ACT_ERRNO`—an error. This prevents containers from mounting filesystems, which could be used to access host resources.”

[SEARCH for “keyctl”]

NARRATION:

“Similarly, `keyctl` is blocked. This syscall manages the kernel keyring and has been exploited in container escapes. By blocking it, we close that attack vector.”

[CLOSE search, scroll to conditional blocks]

NARRATION:

“Docker’s profile also has conditional rules. For example, some syscalls are allowed only with specific arguments. Here’s one for `clone` :”

[HIGHLIGHT:]

```
{
  "names": ["clone"],
  "action": "SCMP_ACT_ALLOW",
  "args": [
    {
      "index": 0,
      "value": 2114060288,
      "op": "SCMP_CMP_MASKED_EQ"
    }
  ]
}
```

NARRATION:

“This says: allow `clone`, but only if argument 0 matches this specific value. This permits normal process creation but blocks dangerous clone operations like creating new namespaces without permission.”

SEGMENT 5: Viewing Active Profiles (9:30 - 11:30)

[SCREEN: Terminal - split screen with two panes]

NARRATION:

“Let’s verify what profile is actually running on a container. I’ll start a container in one terminal and inspect it from the host.”

[LEFT PANE - TYPE:]

```
docker run --name seccomp-test -d ubuntu:22.04 sleep 3600
```

NARRATION:

“We’ve started a container named ‘seccomp-test’ that will sleep for an hour. Now let’s inspect its Seccomp status.”

[RIGHT PANE - TYPE:]

```
docker inspect seccomp-test | grep -i seccomp
```

[PRESS ENTER]

NARRATION:

“Docker’s inspect output doesn’t show the full profile, but we can check from the kernel’s perspective. First, let’s get the container’s main process ID.”

[TYPE:]

```
docker inspect --format '{{.State.Pid}}' seccomp-test
```

[COMMAND RETURNS PID, e.g., 12345]

NARRATION:

“Great, the process ID is 12345. Now we can check the Seccomp status in the proc filesystem.”

[TYPE - replacing 12345 with actual PID:]

```
grep Seccomp /proc/12345/status
```

[OUTPUT shows:]

```
Seccomp:      2
```

NARRATION:

“A Seccomp value of 2 means ‘filtering mode’ is active—a Seccomp profile is being enforced. If this were 0, it would mean no Seccomp. Value 1 is strict mode, which is rarely used.”

[TYPE:]

```
docker stop seccomp-test && docker rm seccomp-test
```

SEGMENT 6: Lab Exercise Introduction (11:30 - 12:30)**[SCREEN: PPT Slide 4 - Lab exercise overview]****NARRATION:**

“Now it’s your turn. In Lab 3.1, you’ll:

1. Start containers with and without Seccomp
2. Test various syscalls to see which are blocked
3. Read and analyze Docker’s default profile
4. Identify specific blocked syscalls and understand why

You’ll experiment with syscalls like `mount`, `unshare`, `keyctl`, and `bpf` to see Seccomp enforcement in action.”

[TRANSITION: PPT Slide 5 - Key takeaways]**SEGMENT 7: Wrap-Up (12:30 - 13:00)****[SCREEN: PPT Slide 5 - Summary points]****NARRATION:**

“Let’s recap what we covered:

- ✓ Seccomp filters system calls at the kernel level
- ✓ Docker applies a default profile that blocks ~50+ dangerous syscalls
- ✓ Blocked syscalls include `mount`, `keyctl`, `unshare`, and many others
- ✓ You can inspect Seccomp status through `/proc` filesystem
- ✓ Seccomp dramatically reduces attack surface even if an attacker gains root

In the next video, we’ll go hands-on building our own custom Seccomp profiles from scratch, tailored to specific applications. This is where Seccomp becomes truly powerful.

See you in the next lesson!”

[FADE OUT]**POST-PRODUCTION NOTES****B-Roll Suggestions:**

- Close-up screen recordings of syscall documentation

- Animated diagram of syscall flow with Seccomp interception
- Overlay graphics showing syscall counts (total vs. blocked)

Audio Notes:

- Add subtle tech/code background music at 10% volume during intro and outro
- Boost audio during terminal segments for clarity
- Add soft “pop” sound effect when commands execute successfully
- Add subtle “error” sound when blocked syscalls fail

Text Overlays:

- Display syscall names in large text when first mentioned
- Show “ALLOWED” or “BLOCKED” labels during demos
- Timer in corner during longer command executions
- Key term definitions as lower-thirds

Timing Checkpoints:

- 0:00-1:30: Intro (under 2 min ✓)
- 1:30-6:30: Concept + Live Demo (5 min)
- 6:30-11:30: Profile Analysis (5 min)
- 11:30-13:00: Lab + Wrap (1.5 min)
- **Total: 13:00** (within 12-14 min target)

Files Needed:

- `slides_module3_video1.pptx` (5 slides)
- `default-seccomp-profile.json` (from Docker GitHub)
- `lab_3.1_instructions.md`
- `lab_3.1_validation.sh` (automated checker)

INSTRUCTOR NOTES

Common Issues:

- If `unshare` works even with default Seccomp, check Docker version (requires 1.12+)
- Some distributions (like Debian) may have different default profiles
- PID retrieval may fail if container exits quickly—use `sleep` as shown

Variation Options:

- Can demonstrate with `podman` as alternative to Docker
- Can show `seccomp-tools` CLI for advanced analysis
- Can briefly show BPF bytecode output (advanced audiences only)

Accessibility:

- Ensure terminal font size shows exactly 20-22 lines
- Use high-contrast theme (light text on dark background)
- Speak commands aloud before typing
- Pause after each command output for 2-3 seconds